



ARTIFICIAL INTELLIGENCE · LECTURE 04+05

Searching Without a Hint

Uninformed Search: Breadth-First, Depth-First & Uniform-Cost



WHERE WE'RE HEADED

Learning Objectives



Explain what “uninformed (blind) search” means, using a real-life example.



Describe a search tree: root, frontier, expanding a node, explored set.



Trace Breadth-First Search (BFS) step by step.



Trace Depth-First Search (DFS) step by step.



Trace Uniform-Cost Search (UCS) and explain why it finds the cheapest path.



Compare the three using completeness, optimality, time and space.

We Have the Map. Now How Do We Walk It?

Last lecture: we learned to describe a problem — states, actions, goal, cost. Today: how does an agent actually FIND the solution?



REAL LIFE: You reach an unfamiliar town with no GPS. Each signboard only tells you which towns are connected to this one — nothing about which direction is “closer” to your destination. How will you search?



BFS

Level by level

Try every nearby road first, then roads further out



DFS

Go deep first

Follow one road as far as it goes, then come back



UCS

Cheapest first

Always continue along the cheapest route so far

What Is “Uninformed” Search?

Uninformed doesn't mean stupid — it means no extra hints.



Uninformed (Blind) Search

The agent knows ONLY the problem definition: it can list actions and test “Is this the goal?” — but has no idea whether a move is getting it closer.

Real life: Looking for your lost phone in a dark house. You check room by room — nobody tells you “warmer” or “colder.”

Today: BFS · DFS · UCS



Informed (Heuristic) Search

The agent gets a hint — an estimate of how close a state is to the goal — and uses it to choose smarter moves.

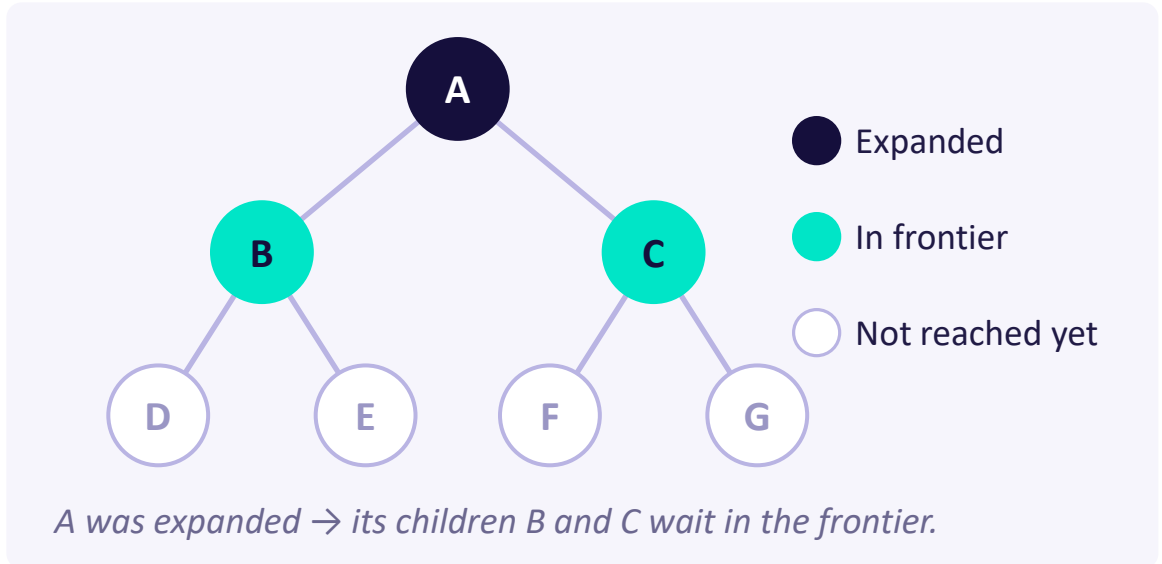
Real life: Your phone's GPS shows an arrow pointing toward the destination, so you head in roughly the right direction.

Next lecture: Informed Search: Heuristics, Heuristics, Greedy Best-First & A*

The Search Tree & The Search Recipe

The state space is a map; searching builds a tree of paths growing out of the start.

Root node	The starting state — where the tree begins.
Expand	List every action from a node and create its child nodes.
Frontier	The “to-do list” of nodes waiting to be explored.
Explored set	Places already visited — so we never walk in circles.
Leaf	A node with no children created yet.



THE SEARCH RECIPE

- 1 Put the start node in the frontier
 - 2 **PICK one node from the frontier**
 - 3 Goal? Stop! Else expand it
 - 4 Add its children to the frontier. Repeat
- Only step 2 changes:** BFS picks the oldest node (queue) · DFS picks the newest node (stack) · UCS picks the cheapest node (priority queue)

Real-life anchor: a delivery app keeps possible routes in the frontier, then repeatedly picks the next route to examine.

4 Yardsticks to Judge Any Search

Before comparing algorithms, we need to agree how to measure them.



Completeness

Will it always find a solution, if one exists?



Optimality

Is the solution it finds the cheapest / best one?



Time

How many nodes does it have to look at?



Space

How many nodes must it keep in memory at once?

THREE NUMBERS WE USE TO DESCRIBE SIZE



Choices at each step
(branching factor)



Depth of the nearest goal
(how many steps away)



Longest possible path
(maximum depth)

Real-life anchor: judge a delivery route by four questions — will it find one, is it best, how long, and how much memory?

Breadth-First Search (BFS)

Explore everything close first — level by level.

THE IDEA

Check everything 1 step from the start, then everything 2 steps away, then 3 ... never go deeper until the current level is finished.

REAL LIFE

Finding a friend in a 3-floor mall: you check every shop on floor 1, then every shop on floor 2, then floor 3. Same idea: a ripple spreading on water, or “friends → friends of friends” on social media.

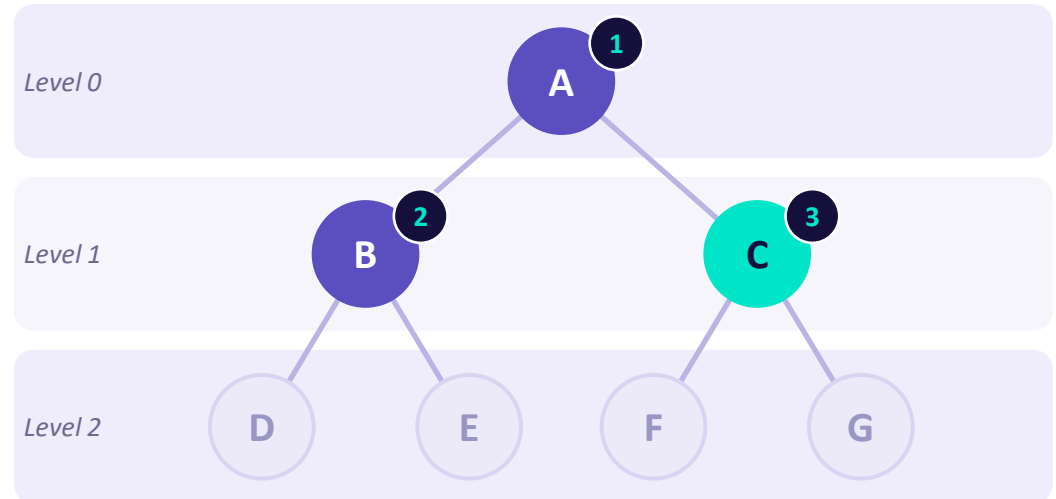
HOW IT IS BUILT

The frontier is a QUEUE — first in, first out. The oldest node always goes next.

✓ Finds the path with the FEWEST steps

✗ Needs a LOT of memory

Goal = C

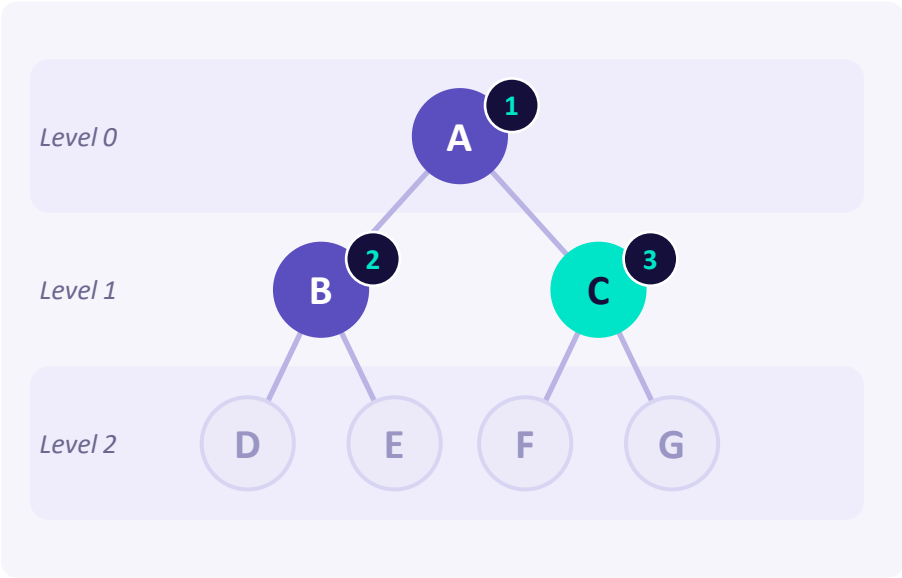


BFS checked 3 nodes: A → B → C

Badges show the order nodes were checked. It finishes each level before starting the next.

BFS Trace: Tree, Goal = C

The frontier is a queue — take from the FRONT, add new nodes at the BACK.



Badges show the order nodes are checked: $A \rightarrow B \rightarrow C$.

QUEUE = ticket line. The person who arrived first is served first (first in, first out).

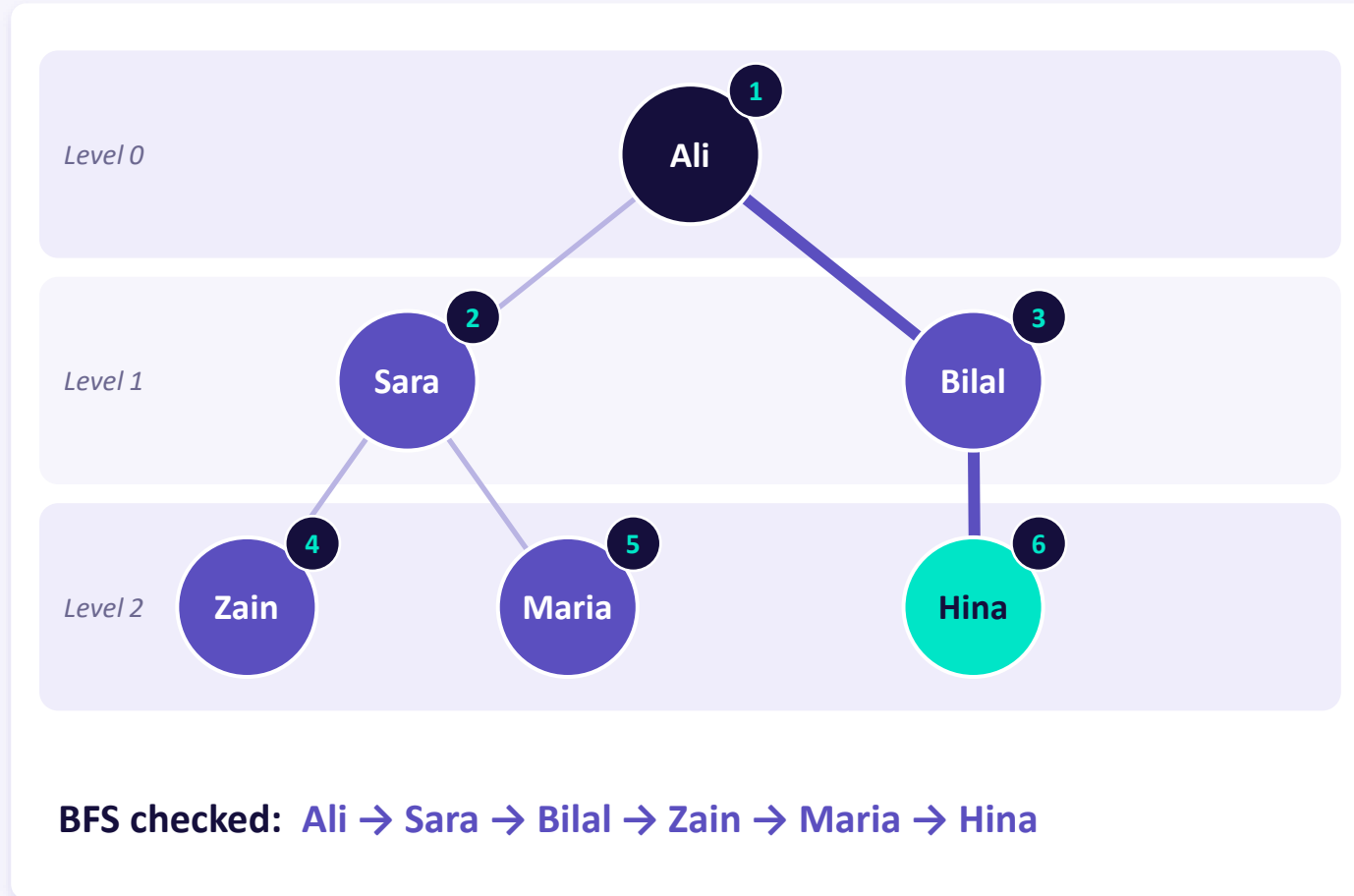
Step	Take from front	Goal?	Add to back	Queue now
1	A	No	B, C	[B, C]
2	B	No	D, E	[C, D, E]
3	C	YES — goal!	-	-

- Step 1.** A leaves the queue. Its children B and C join the back.
- Step 2.** B is at the front, so B goes next. D and E join the back — behind C.
- Step 3.** C is now at the front and it is the goal. 3 nodes checked.

Why C before D and E? C joined the queue earlier. That is what makes BFS go level by level.

BFS in Real Life: Friend Chains

Shortest chain of friends from Ali to Hina — BFS checks friends, then friends of friends.



THE QUESTION

Ali's friends: Sara, Bilal. Sara's friends: Zain, Maria. Bilal's friend: Hina. What is the shortest friend chain from Ali to Hina?

HOW BFS ANSWERS

Level 1 = Ali's direct friends. Level 2 = their friends. Hina appears at level 2, reached through Bilal.

WHERE YOU SEE IT

LinkedIn's "2nd connection", "people you may know", and the famous "six degrees of separation".

✓ Answer: Ali → Bilal → Hina (2 hops)

Depth-First Search (DFS)

Go as deep as possible; only come back when you hit a dead end.

THE IDEA

Pick one branch and keep going down it. At a dead end, back up to the last junction and try the next branch (this is called backtracking).

REAL LIFE

Exploring a maze: you follow one corridor all the way to its end, hit a wall, return to the last turning and try another. Same idea: the Wikipedia rabbit hole — one link leads to another, then another.

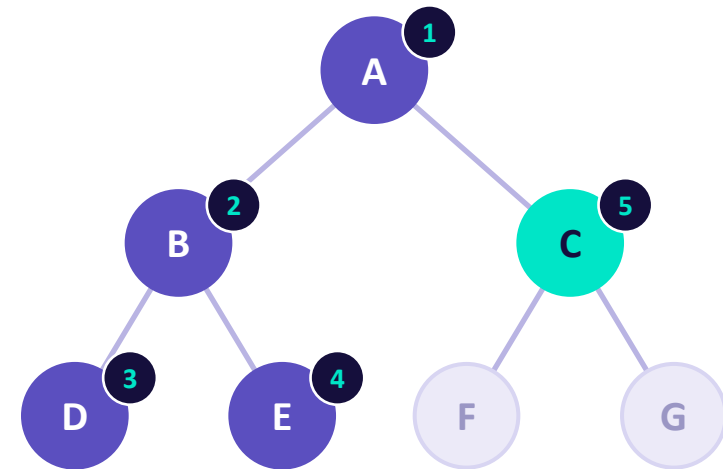
HOW IT IS BUILT

The frontier is a STACK — last in, first out. The newest node always goes next.

✓ Uses very LITTLE memory

✗ Can get lost; not optimal

Goal = C

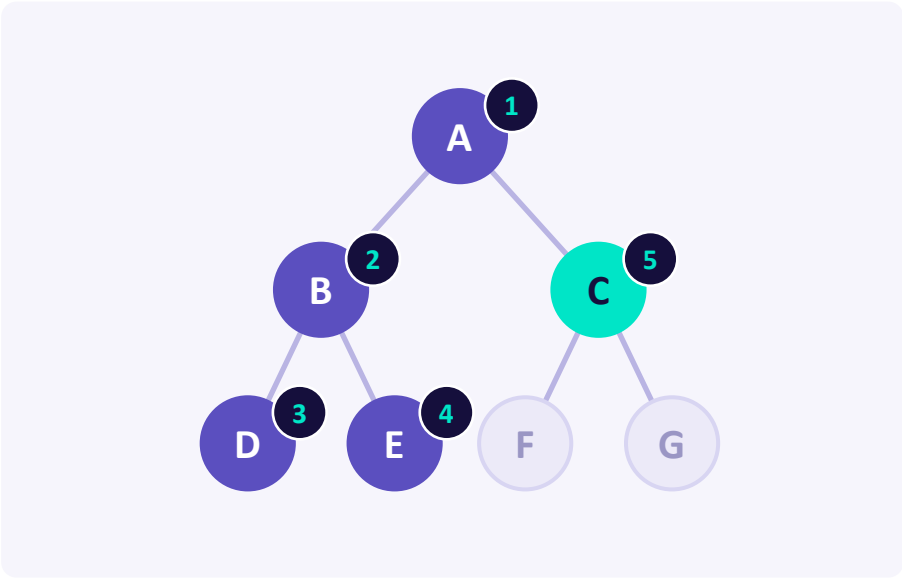


DFS checked 5 nodes: A → B → D → E → C

It explored all of B's branch first. If the goal had been D, DFS would have won — the winner depends on where the goal is.

DFS Trace: Tree, Goal = C

The frontier is a stack — the NEWEST node is always taken first.



Badges show the order nodes are checked: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$.

STACK = pile of plates. The plate placed last is picked up first (last in, first out).

Step	Pop from top	Goal?	Push (left on top)	Stack now (top → bottom)
1	A	No	B, C	[B, C]
2	B	No	D, E	[D, E, C]
3	D	No	(dead end)	[E, C]
4	E	No	(dead end)	[C]
5	C	YES — goal!	-	-

Steps 3 and 4. D and E have no children: dead ends. Nothing is pushed, so the stack shrinks. **This is backtracking.**

Step 5. Only C is left on the stack. It is the goal. 5 nodes checked (BFS needed). **Pink rows = dead ends.** The stack drops back to the last unexplored branch automatically.

Uniform-Cost Search (UCS)

When steps cost different amounts, always continue the cheapest path so far.

THE IDEA

Always expand the node whose total cost so far — written g — is the lowest. We care about the total bill, not the number of steps.

REAL LIFE

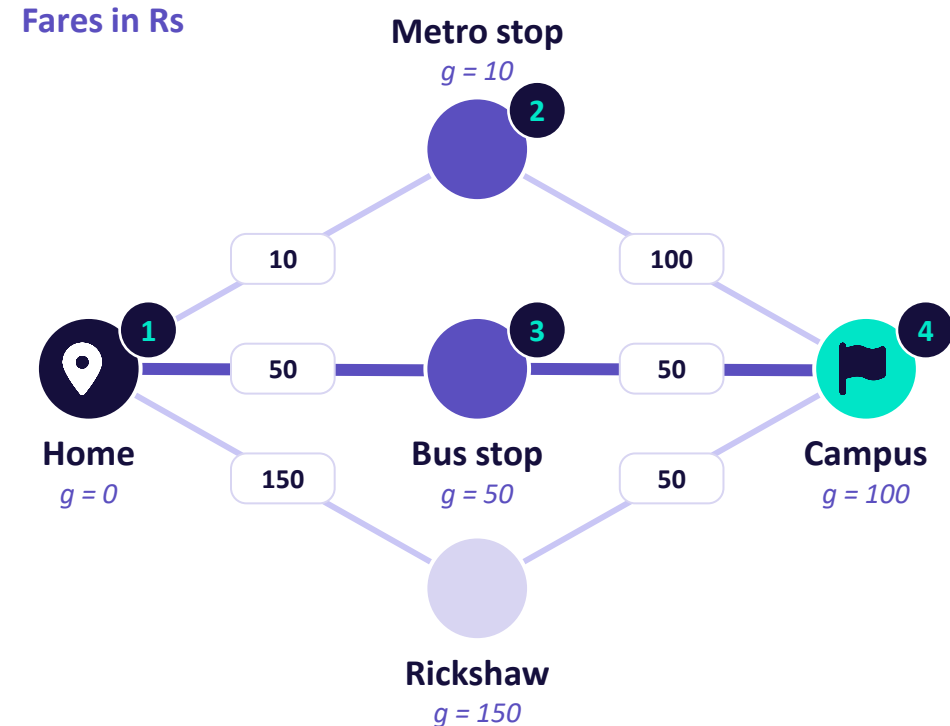
Getting from home to campus by metro, bus or rickshaw: you add up the full fare for each option, then take the cheapest — not the one with the fewest stops.

HOW IT IS BUILT

The frontier is a PRIORITY QUEUE — the cheapest path is always served first.

✓ Finds the CHEAPEST path when costs are positive

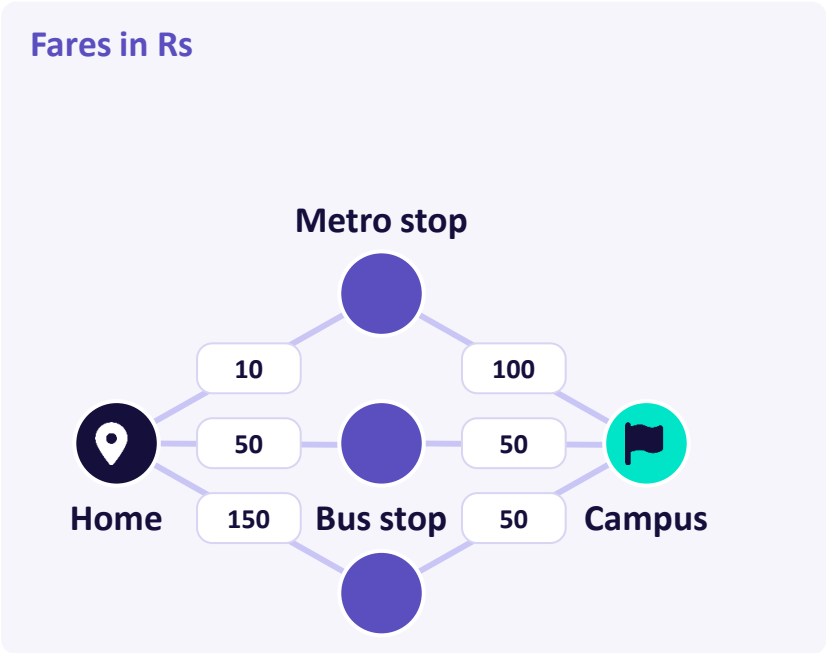
✗ Can explore wrong-way paths



Cheapest: Home → Bus → Campus = Rs 100. *The metro path costs Rs 110, so UCS never picks it — yet BFS (counting steps only) might.*

UCS Trace: Home → Campus

The frontier is a priority queue — the lowest total cost so far (*g*) is always taken first.



PRIORITY QUEUE = emergency room. The most urgent (here: cheapest) is served first.

Step	Pop cheapest (<i>g</i>)	Goal?	What we do with its neighbours	Queue now (node <i>g</i>)
1	Home (0)	No	add Bus stop: $0+50 = 50$; add Metro stop: $0+10 = 10$; add Rickshaw: $0+150 = 150$	[Metro stop 10, Bus stop 50, Rickshaw 150]
2	Metro stop (10)	No	add Campus: $10+100 = 110$	[Bus stop 50, Campus 110, Rickshaw 150]
3	Bus stop (50)	No	update Campus: $110 \rightarrow 100$ (cheaper)	[Campus 100, Rickshaw 150]
4	Campus (100)	YES — goal!	-	-

Do not stop early! Campus appeared at step 2 with $g = 110$, but UCS did not stop. At step 3 a cheaper route ($g = 100$) was found. Only when Campus is the cheapest node in the queue (step 4) is it confirmed.

BFS vs. DFS vs. UCS — Side by Side

Same recipe, different “pick next” rule — very different behaviour.

	BFS	DFS	UCS
Picks next...	Oldest node (queue)	Newest node (stack)	Cheapest node so far (priority queue)
Always finds a solution?	Yes (if finite choices)	No — can dive down an endless path	Yes (if every step costs at least a small positive amount)
Finds the best one?	Only if all steps cost the same	No	Yes — the cheapest path
Time	b^d — grows very fast	b^m — can be very slow	Large — grows with solution cost
Memory	b^d — huge	$b \times m$ — tiny	Large — like BFS
Real-life picture	Floor-by-floor mall search	Following one corridor to its end	Picking the cheapest total fare
Best used when...	Fewest steps matter and the problem is small	Memory is tight, or solutions are deep	Steps have different costs

Real-life anchor: BFS = nearby first, DFS = one corridor deep, UCS = cheapest total fare first.

■ AVOID THESE TRAPS

Watch Out! Common Search Pitfalls



BFS is not the same as cheapest.

It finds the path with the fewest STEPS. When roads cost different amounts, a cheaper route with more steps can exist.



DFS can get lost.

It may follow an endless path — or bounce between two towns forever — and never return to a goal that was close by.



Remember where you have been.

Without an explored set, a search walks in circles and repeats the same cities again and again.



Memory runs out before time does.

With 10 choices per step, BFS must hold about 1 million nodes at depth 6 and about 10 billion at depth 10.



Blind means slow.

Uninformed search explores in every direction. That is exactly why we need hints (heuristics) — the topic of the next lecture.

Real-life anchor: a GPS that ignores road cost may choose a short-looking route that is actually slow or expensive.

Where You Meet These Algorithms Every Day

Every one of these starts the same way: formulate the problem, then choose a search strategy.



- Web crawlers: visit a page, then every page it links to, level by level
- Social networks: “friends of friends” and degrees of separation
- Fewest metro stops between two stations



- Maze and Sudoku solvers: try, hit a wall, backtrack
- Exploring folders inside folders on your computer
- Checking move sequences in puzzles and games (with a depth limit)



- Route planners: the cheapest or fastest road, not the fewest turns (Dijkstra's algorithm is a close cousin of UCS)
- Internet routers choosing the lowest-cost path for your data
- Cheapest-fare finders for buses and flights



Quick Quiz

Answer on paper or by raising your hand: A, B or C

Q1 BFS explores the search tree...

- A) Deepest path first B) Level by level C) Cheapest path first

Q2 Which algorithm uses a stack (last-in, first-out)?

- A) UCS B) BFS C) DFS

Q3 Roads cost different amounts. Which one is guaranteed to find the cheapest route?

- A) UCS B) BFS C) DFS

Q4 Which algorithm needs the LEAST memory?

- A) BFS B) DFS C) UCS

Q5 Fewest “friend hops” between two people on a social network — best fit?

- A) DFS B) UCS C) BFS

Q6 True or False: DFS can get stuck on an endless path and never find a nearby goal.

- A) True B) False



Answers & Why

Q1 BFS explores the search tree...

B) Level by level — BFS checks everything 1 step away, then 2 steps, and so on.

Q3 Roads cost different amounts. Which one is guaranteed to find the cheapest route?

A) UCS — UCS expands the lowest total cost. BFS only counts steps.

Q5 Fewest “friend hops” between two people on a social network — best fit?

C) BFS — Every hop costs the same, so level-by-level BFS gives the fewest hops.

Q2 Which algorithm uses a stack (last-in, first-out)?

C) DFS — DFS always takes the newest node — that is what sends it deep.

Q4 Which algorithm needs the LEAST memory?

B) DFS — DFS remembers only the current path and its unexplored siblings.

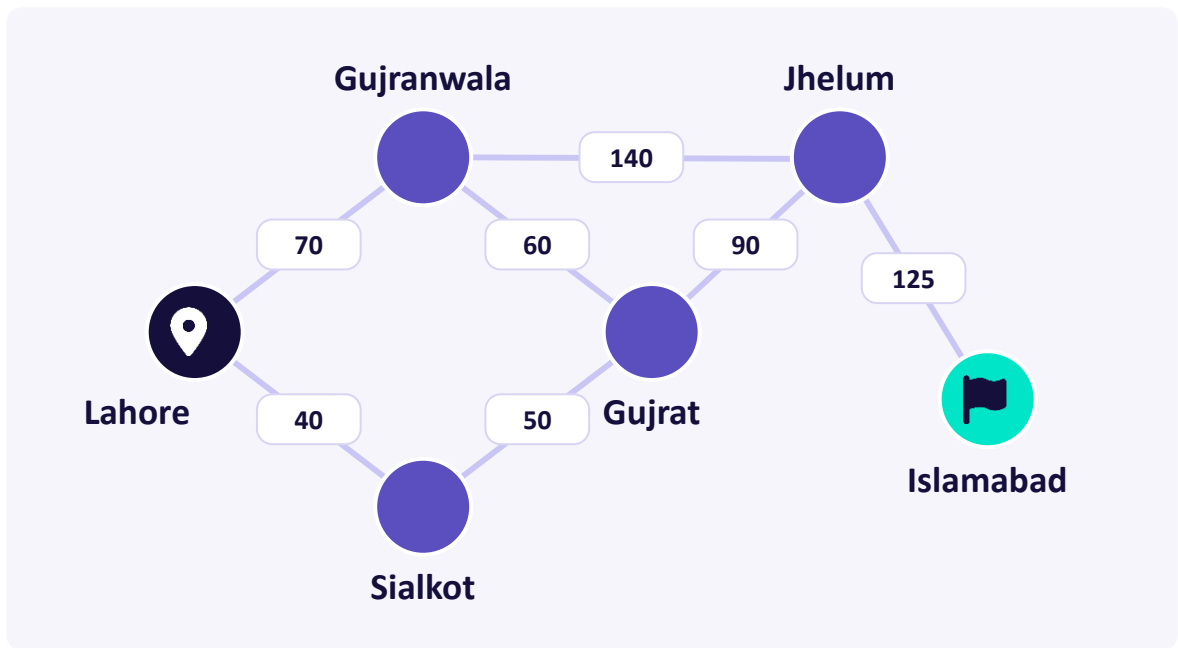
Q6 True or False: DFS can get stuck on an endless path and never find a nearby goal.

A) True — Without a depth limit or an explored set, DFS may dive forever or loop.

Trace the Search: Lahore → Islamabad

Costs are simplified distances in km. When you have a choice, go in alphabetical order. Start = Lahore, Goal = Islamabad.

- 1 **Trace BFS.** Write the order cities are checked, and the final path.
- 2 **Trace DFS.** Same thing — order checked, and final path.
- 3 **Trace UCS.** Write each city with its cost so far (g) in the order it is expanded.
- 4 **Fill the table.** Path, number of steps and total km for each. Which is cheapest? Which has the fewest steps?
- 5 **Check with ChatGPT.** Paste the road list, ask it to trace all three, and compare. Did it make any mistakes?



Roads (km): Lahore–Gujranwala 70 · Lahore–Sialkot 40 · Gujranwala–Gujrat 60 · Gujranwala–Jhelum 140 · Sialkot–Gujrat 50 · Gujrat–Jhelum 90 · Jhelum–Islamabad 125

Algorithm	Path found	Steps	Total km
BFS			
DFS			
UCS			

Three Algorithms, Three Different Answers



BFS

Order checked: Lahore → Gujranwala → Sialkot → Gujrat → Jhelum → Islamabad

Path: Lahore → Gujranwala → Jhelum → Islamabad

3 steps · 335 km

Fewest steps, but not the cheapest



DFS

Order checked: Lahore → Gujranwala → Gujrat → Jhelum → Islamabad

Path: Lahore → Gujranwala → Gujrat → Jhelum → Islamabad

4 steps · 345 km

Found a path — neither fewest steps nor cheapest



UCS

Order checked: Lahore (0) → Sialkot (40) → Gujranwala (70) → Gujrat (90) → Jhelum (180) → Islamabad (305)

Path: Lahore → Sialkot → Gujrat → Jhelum → Islamabad

4 steps · 305 km

The cheapest path — optimal

Lesson: only UCS is guaranteed cheapest here, because the roads have different costs. BFS and DFS ignore cost.

Key Takeaways

- Search is how an agent finds a solution once the problem is formulated: keep expanding nodes from the frontier until the goal appears.
- Uninformed (blind) search has no hints about the goal — only the problem definition.
- BFS goes level by level: it finds the fewest-steps path, but it eats memory.
- DFS goes deep first: very light on memory, but it can get lost and is not optimal.
- UCS expands the cheapest path first: it finds the cheapest solution when step costs are positive.
- Judge any search with four yardsticks: completeness, optimality, time and space.

Real-life anchor: after formulation, BFS/DFS/UCS are different strategies for walking through the same problem map.



Recommended Reading



Primary Text

Russell, S. & Norvig, P. (2010). Artificial Intelligence: A Modern Approach (3rd ed.). Chapter 3, Sections 3.3–3.4 (Searching for Solutions; Uninformed Search Strategies).



Reference

Luger, G.F. & Stubblefield, W.A. (2009). AI Algorithms, Data Structures, and Idioms in Prolog, Lisp, and Java — chapters on state-space search.

NEXT LECTURE

Informed Search: Heuristics & Greedy Best-First Search